

Research Article

PPDRL: A Pretraining-and-Policy-Based Deep Reinforcement Learning Approach for QoS-Aware Service Composition

Kan Yi,¹ Jin Yang¹,² Shuangling Wang,¹ Zhengtong Zhang,² and Xiao Ren²

¹Science and Technology on Information Systems Engineering Laboratory, Xi'an, China

²School of Computer Science and Technology, XiDian University, Xi'an, China

Correspondence should be addressed to Jin Yang; yangjin@stu.xidian.edu.cn

Received 8 November 2021; Revised 23 March 2022; Accepted 24 March 2022; Published 22 April 2022

Academic Editor: Shah Nazir

Copyright © 2022 Kan Yi et al. This is an open access article distributed under the Creative Commons Attribution License, which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited.

Service composition is a mainstream paradigm for rapidly constructing large-scale distributed applications. QoS-aware service composition, i.e., selection of the optimal execution plan that maximizes the composition's end-to-end QoS properties, is an active area of research and development endeavors in service composition. In this paper, we propose PPDRL, a pretraining-and-policy-based deep reinforcement learning approach, to solve the QoS-aware service composition problem. Its significant feature is to incorporate a maximum likelihood estimate and a policy scoring mechanism into a deep reinforcement learning framework. As a result, our approach can balance the exploitation and exploration efforts adaptively and can search for the solution space in a robust and efficient manner. We have executed our approach to solve 6 randomly generated QoS-aware service composition problems with different sizes and structures based on QWS data set including 2,507 real Web services classified into 233 categories. The results indicate that our approach can find near-optimal solutions within moderate numbers of iteration and has performance superiority in comparison with five state-of-the-art algorithms.

1. Introduction

Modern enterprises require an efficient and flexible scheme for pooling globally available services together to quickly adapt to various customer needs and dynamic market conditions. It has been proved that service composition becomes a convincing computing paradigm for rapidly developing large-scale distributed applications by these services within and across organizational boundaries.

Over the past decade, services composition has become a prevalent area of academic efforts, with a large amount of research work produced [1–7]. Among these works, QoS-aware service composition, i.e., selection of the optimal execution plan that maximizes the composition's end-to-end QoS properties, is one of the most active areas of research and development endeavors.

QoS-aware service composition is usually modeled as a combinatorial optimization problem in many previous studies [8]. Existing service composition methods include classic algorithm, heuristic algorithms, and learning-based

algorithm, as discussed in Section 2. Among them, learning-based algorithm has received much recent attention. The key idea of learning-based algorithm is to construct a QoS-value prediction model using training samples of different solutions and then explore better solutions using some searching algorithms. Among these algorithms, the deep reinforcement learning (DRL) based algorithms have received the most attention, and this is because DRL-based algorithms can solve the large-scale service composition problem adaptively and can adapt to the change of environment automatically [3]. Although previous studies on DRL-based algorithms show promising results, one critical drawback is that it requires a lot of time to train a useful model. Apparently, the assumption of the abundance of time used in many DRL-based methods is invalid considering the constrained time for optimization in the real-world problems. Given the limited optimization time in practice, only a limited number of iteration can be obtained. It typically results in a less accurate model that jeopardizes the exploration for better solutions.

In this paper, we propose PPDRL, a pretraining-and-policy-based deep reinforcement learning approach, to solve the QoS-aware service composition problem. Its key idea is to incorporate a maximum likelihood estimate and a policy scoring mechanism into a deep reinforcement learning framework. As a result, PPDRL can solve the large-scale service composition problems adaptively and can adapt to the change of environment automatically.

In summary, our work makes the following contributions:

We introduce and formulate the QoS-aware service composition (QSC) problem and model it as a *combinatorial optimization* problem.

We propose PPDRL, a novel approach to address QSC problem, from an unconventional direction. PPDRL can recommend promising solution by combining the maximum likelihood estimate and the policy scoring mechanism into a deep reinforcement learning framework.

We investigate the effectiveness of the proposed algorithm by solving 6 composite service instances with different sizes and structures. The experiment results have also shown the performance superiority of the proposed algorithm in comparison with five state-of-the-art algorithms.

The rest of this paper is organized as follows. Section reviews some related work. Section defines quality criteria for atomic services and the composite service model, and then gives the problem formulation. Section describes our PPDRL for QoS-aware service composition problem in detail. Section introduces our experiment setups and reports the performance evaluation and comparison of different algorithms. The paper is finally concluded in Section.

2. Related Work

QoS-aware service composition has received much attention from both industry and academia. We can classify the previous studies into three categories: classic algorithms, heuristic algorithms, and learning-based algorithms, where the last one is most recent and relevant to our work.

2.1. Classic Algorithms. Many classic algorithms have been proposed to solve the QoS-aware service composition problem, such as integer programming [9], backtracking, and branch-and-bound. Wang et al. [10] proposed a branch-and-bound algorithm based on multilevel graph that finds a feasible path and maximizes the utility of the path while reducing the search space. Fan et al. [11] presented a chained dynamic programming and hybrid pruning algorithm to transform the composition task into an equivalent task with reduced computational complexity. White et al. [12] proposed a collaborative filtering algorithm based on matrix factorization that allows programs to automatically adapt to QoS changes in their component services. Wakrime et al. [13] proposed a formal method based on satisfiability to model and verify Web service composition. Chattopadhyay

et al. [4] presented an abstraction refinement-based approach to reduce the search space. Yan et al. [14] proposed a method to combine the system search algorithm with the planning algorithm. Mirandola et al. [15] proposed a set of software metrics that quantify the adaptability of service-oriented applications.

Classic algorithms use different deterministic models to solve the service composition problem. These algorithms have lower time complexity but may contain strict preconditions that may not hold in application scenarios, which severely impairs its performance.

2.2. Heuristic Algorithms. Jatoth et al. [1] proposed a MapReduce-based evolutionary algorithm with guided mutations applied in large service environments. Bhushan et al. [16] proposed a hybrid particle swarm optimization technique that combines particle swarms and fruit flies for search and optimization. Rodriguez-Mier et al. [17] presented a hybrid local global search method to extract the optimal QoS value with the least number of services. Siriweera et al. [18] proposed a customizable transaction and QoS-aware service selection approach. Boussalia et al. [19] proposed an approach based on the use of a new Extended Bat Inspired Algorithm. Hammas et al. [20] proposed an architecture that supports dynamic combination and global QoS optimization. Wang et al. [21] conducted a configurability study on the artificial bee colony algorithm (ABC) and implemented a prototype system for configuring ABC parameters and optimization strategies. Da Silva et al. [2] proposed a method that generates a composition scheme based on information stored in a graph database and optimized it with genetic algorithm. Wu et al. [22] studied the transaction attributes of services and optimized the problem by ant colony algorithm. Klein et al. [23] proposed a heuristic approach based on Hill-Climbing that reduces the time complexity by narrowing the search space. Canfora et al. [24] proposed a classical genetic algorithm to provide a more stable algorithm output and get better solutions. As moveable users are very common in current service composition, frequent service re-configurations are carried out after the initial provision to maintain the QoS values, as is studied in [25, 26].

Heuristic algorithms are usually based on evolutionary algorithms. These algorithms are often bound to find a local optimal solutions. However, they have high time complexity and are usually designed only for offline service composition tasks.

2.3. Learning-Based Algorithms. In recent years, a large number of learning algorithms have been applied to solve service composition problems. Wang et al. [3] proposed a deep Q-learning (DQN) algorithm based on LSTM and fully connected layer. Labbaci et al. [27] proposed a deep learning approach for long-term QoS. Kazem et al. [28] used Bayesian network to predict new values of certain QoS attributes. Wang et al. [29] proposed a Q-learning algorithm that uses a Gaussian process to predict Q-valued functions. Wang et al. [30] proposed a multiagent algorithm based on Sarsa to

achieve the maximum possible benefit. Wang et al. [31] proposed an automatic layered reinforcement learning method that replaces manual generation of task graph by systematically integrating automatic task decomposition. Jungmann et al. [32] proposed a recommendation mechanism to expand state-space-based service composition. Elsayed et al. [33] proposed a new method combining Genetic Algorithm (GA) and Q-learning. Zhao et al. [34] proposed a machine learning method using learning-to-rank algorithm to automatically learn user preferences. Shehu et al. [35] used a learning automata-based non-negative matrix factorization algorithm (LANMF) to predict network delay and optimized QoS-aware service composition problems through four evolutionary algorithms. Wang et al. [36] proposed a Q-learning-based algorithm based on Markov decision process. Li et al. [37, 38] model the problem as a Markov decision process (MDP) and propose a hierarchical deep reinforcement learning (DRL) model based on graph neural network (GNN) in [37]. Simulations experiments demonstrate their performance in virtual network function service chaining.

Learning-based algorithms use neural network to learn the optimal strategy. Although these algorithms require a large amount of time to train the models, they may potentially find a better solution due to its model complexity.

3. Problem Statement

This paper focuses on the QSC problem. It aims at finding the best set (in terms of QoS) of atomic services to execute the abstract tasks defined in a composite service. In this section, we first present the quality criteria in the context of atomic services and provide a brief definition for each criterion. After that, we construct the composite service model to facilitate a mathematical formulation of QSC problem.

3.1. Quality Criteria for Atomic Services. We consider six quality criteria for atomic services [5] in this paper, as shown in Table 1:

3.2. Composite Service Model. A composite service cs can be modeled as a directed acyclic graph $cs = (S, E)$, where vertex set $S = \{s_1, s_2, \dots, s_n\}$ consists of many atomic services, and each directed edge $e_{i,j} \in E$ denotes the dependency between a pair of the adjacent atomic services s_i, s_j . To construct a composite service, atomic services need to be connected by different structures. In this paper, we consider four service composition structures: sequence, concurrency, condition, and loop, as suggested in [8].

Each atomic service $s_i \in S$ must belong to the one and only one *service class* [39], and different atomic services may belong to the same service class. A service class is a collection of candidate atomic services with a common functionality but different QoS properties. The QoS-aware service composition problem is to select one service candidate from each service class for each atomic service to construct a composite service, so that it can maximize the six QoS properties.

Mathematically, given a composite service cs containing n atomic services $S = \{s_1, s_2, \dots, s_n\}$. Let cs have m service classes $C = \{C_1, C_2, \dots, C_m\}$, and each service class C_i contains k_i different service candidates: $C_i = \{c_1, c_2, \dots, c_{k_i}\}$. The QSC problem can be formulated as follows:

$$\max_{\mathbf{x}} (\text{QoS}(cs, \mathbf{x})), \quad (1)$$

where (1) states that the goal of QSC problem is to maximize the QoS value for a given composite service cs . $\mathbf{x} = \{x_1, x_2, \dots, x_n\}$ denotes the service candidate selecting result for each atomic service in cs . Note that there is one and only one service candidate can be selected for each atomic service in cs . The constraints of the problem [39] are as follows:

$$\begin{aligned} (q_p(cs, \mathbf{x}), q_d(cs, \mathbf{x}), q_a(cs, \mathbf{x}), q_t(cs, \mathbf{x}), q_s(cs, \mathbf{x}), q_r(cs, \mathbf{x})) \\ \leq (Q_p^c, Q_d^c, Q_a^c, Q_t^c, Q_s^c, Q_r^c). \end{aligned} \quad (2)$$

$$\sum_{j=1}^{|c(s_i)|} x_{i,j} \quad i = 1, 2, \dots, n. \quad (3)$$

$$x_{i,j} \in \{0, 1\} \quad i = 1, 2, \dots, n; \quad (4)$$

$$j = 1, 2, \dots, \max_{i=1}^n (|c(s_i)|), \quad (5)$$

where (6) state that each QoS attribute must meet the predefined QoS constraints. (3) and (5) guarantee that there is one and only one candidate service selected for each atomic service in cs .

The goal is calculated by the weighted sum of the six QoS criterias:

$$\begin{aligned} \text{QoS}(cs, \mathbf{x}) &= \alpha_1 \cdot \frac{q_p(cs, \mathbf{x}) - q^{\min_p}(cs)}{q^{\max_p}(cs) - q^{\min_p}(cs)} \\ &= \alpha_2 \cdot \frac{q_d(cs, \mathbf{x}) - q^{\min_d}(cs)}{q^{\max_d}(cs) - q^{\min_d}(cs)} \\ &= \alpha_3 \cdot \frac{q_a(cs, \mathbf{x}) - q^{\min_a}(cs)}{q^{\max_a}(cs) - q^{\min_a}(cs)} \\ &= \alpha_4 \cdot \frac{q_t(cs, \mathbf{x}) - q^{\min_t}(cs)}{q^{\max_t}(cs) - q^{\min_t}(cs)} \\ &= \alpha_5 \cdot \frac{q_s(cs, \mathbf{x}) - q^{\min_s}(cs)}{q^{\max_s}(cs) - q^{\min_s}(cs)} \\ &= \alpha_6 \cdot \frac{q_r(cs, \mathbf{x}) - q^{\min_r}(cs)}{q^{\max_r}(cs) - q^{\min_r}(cs)}. \end{aligned} \quad (6)$$

The QSC problem has been well studied and often modeled as a combinatorial optimization problem and is considered to be NP complete. The NP completeness proof

TABLE 1: Six quality criteria for an atomic service. s

Quality criterion	Description
Execution price: $q_p(s)$	$q_p(s)$ represents The fee that a service requester has to pay for invoking s .
Execution duration: $q_d(s)$	$q_d(s)$ measures the expected delay in seconds between the moment when a request is sent and the moment when the results are received.
Availability: $q_a(s)$	$q_a(s) = I_s/I_t$, where I_s is the number of successful invocations of s and I_t is the number of total Invocations of s , during a constant time interval set by an administrator of the service provider.
Throughput: $q_t(s)$	$q_t(s) = I_t/\theta$, where I_t is the number of total invocations of s and θ is the given time interval.
Successful execution rate: $q_s(s)$	$q_s(s) = N_c(s)/I_t$, where $N_c(s)$ is the number of times that the service s has been successfully completed within the maximum expected time frame and I_t is the total number of invocations.
Reliability: $q_r(s)$	$q_r(s) = 1 - N_e(s)/N_t(s)$, where $N_e(s)$ is the number of error response messages and $N_t(s)$ is the total response messages.

by restriction is established in [8]. The above complexity analysis of QSC rules out the existence of any polynomial-time optimal solution unless $P = NP$. Therefore, we shall focus on the design of a approximation approach to this optimization problem.

4. Pretraining-and-policy-Based Deep Reinforcement Learning for QSC Problem

Our proposed pretraining-and-policy-based deep reinforcement learning (PPDRL) is a hybrid framework for MOPs that combines the pretraining-based strategy and the policy-based deep reinforcement learning method. The key idea behind PPDRL is to incorporate a maximum likelihood estimation (MLE) method and a policy scoring mechanism into a deep reinforcement learning framework.

The overall framework of our proposed approach is shown in Figure 1. PPDRL consists of three important components: initialization, pretraining, and RL training. In the *initialization* module, we use random sampling to generate 50,000 sets of service composition results from a well-known data set and pick the best 64 service composition results as the initial samples. After that, the *pretraining* module trains the actor through MLE to learn the distribution characteristics of good solutions. The actor network is composed of an embedding layer and an rnn layer, to encode the candidate atomic service and output a probability distribution over candidates. After that, the *RL-training* module is called to further train the actor through gradient descent in order to find a combination that can obtain the better QoS values, the detailed process is states in Algorithm 1. During this optimization process, we constantly update the optimal QoS values by continually updating the sample set, and repeating the pretraining and RL-training process until the convergence condition is met.

4.1. Maximum Likelihood Estimate. We use MLE here as a statistical method to find the parameters of the correlation probability density function of a sample set. Specifically, the mathematical principle of MLE is to maximize the probability density function [40], which is defined as follows:

$$\text{lik}(\theta) = f_D(x_1, x_2, \dots, x_n | \theta). \quad (7)$$

As shown in (7), given a probability distribution D , a distribution parameter θ , and n values $x_1, x_2, \dots, x_n$ sampled

from distribution D , find the θ that maximizes the probability density function.

4.2. Scores-Based Sampling Strategy. For a service composition problem, PPDRL uses the neural network to score the candidate atomic services for each atomic service. Then it samples the candidate service set for each service multiple times based on the scores to prepare for the mini-batch gradient descent. This approach balances exploration and utilization and improves the efficiency of the algorithm.

4.3. Gradient Descent. Gradient descent is the most commonly used method to optimize neural networks, and its calculation process is to solve the minimum value along the direction of gradient descent. In this paper, we use mini-batch gradient descent to optimize the algorithm to avoid falling into local optimum. Mini-batch gradient descent each time select a part of the sample to train the network, this method overcomes the instability of stochastic gradient descent and the inefficiency of batch gradient descent.

4.4. PPDRL Algorithm. Using the optimization process shown in Figure 1, we now discuss the PPDRL algorithm in Algorithm 1. We first define the state, action, and reward of the algorithm. The state is composed of the DAG of the given composite service cs , current service class C_i , and its corresponding service candidates $C_i = \{c_1, c_2, \dots, c_{k_i}\}$. The action is to choose one atomic service from the candidates $a_i = 1, 2, \dots, k_i$ and the reward is the QoS goal estimated by fill the unassigned service by the median QoS value. The agent chooses an atomic service for a current service class in the topological order and receive a feedback. This procedure will proceed until all the service class of the given composite service is assigned a corresponding service.

In the PPDRL algorithm, we use MLE to pretrain the neural network in order to accelerate convergence and use policy-based deep reinforcement learning to find better QoS values. Our objective function is the expected value of QoS in the initial state. So the gradient is the derivative of the objective function to the network parameters, θ , which is defined as follows:

$$\nabla_{\theta} J(\theta) = E_{\pi_{\theta}} [QoS(\pi_{\theta}|s_0) \nabla_{\theta} \log p(\pi_{\theta}|s_0)]. \quad (8)$$

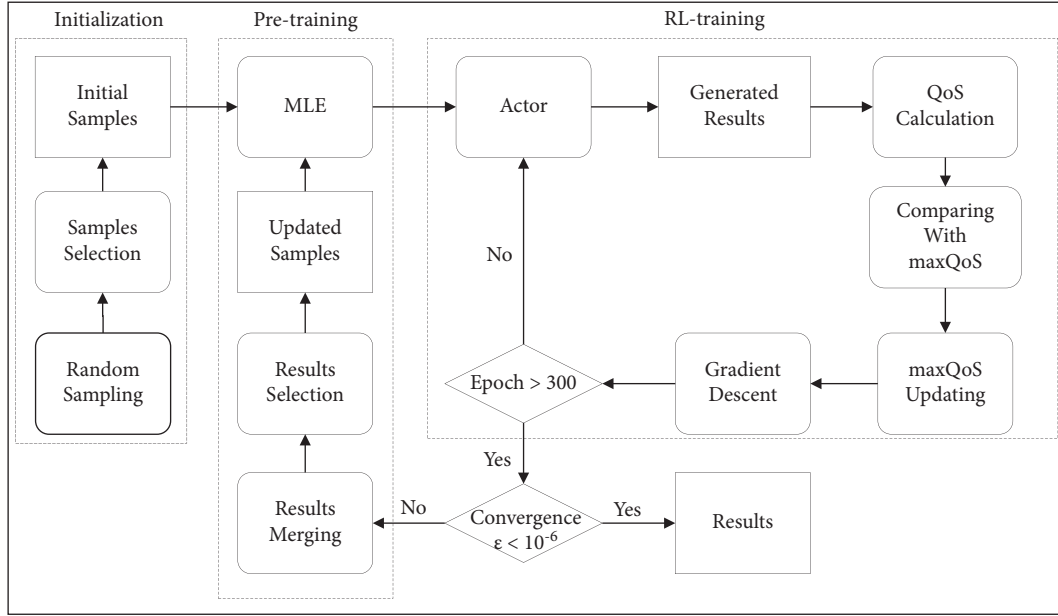


FIGURE 1: Ppdrl overview.

After determining the gradient, we use Adam optimizer [41] to optimize the network parameters and get the optimized QoS value. Finally, we will choose better results to update the sample set and proceed to the next round of training until the algorithm meets the convergence conditions.

5. Experiments

In this section, we examine the performance of PPDRL-SC by empirically comparing it with five state-of-the-art algorithms. The source code and the data can be found in <https://github.com/xdbdilab/ppdrl>.

5.1. Experimental Settings

Running Environment. All experiments run on a server equipped with two 8-core Intel XeonE5-2650v2 2.6 GHz processors, 256GiB RAM, 1.5 TB disk, and running CentOS 7.5.

Workloads. We use a well-known synthetic workflow generator [42] to randomly generate composite service instances with six different sizes (number of atomic services): 10, 30, 50, 70, 90, and 100. For each instance, every algorithm is executed in a large number of iterations and then is forced to stop when converges. For each run in our experiments, each workload is executed for five times and calculate the average of these five runs.

Benchmark. We conduct extensive experiments on the QWS data set [43] that includes 2,507 real Web services. To generate the candidate service set for each atomic service, we

apply a simple text clustering method to all these service names in the QWS data set and produce 233 service classes [39]. The number of candidate services in each service class ranges from 2 to 128.

Performance metrics. We consider two performance metrics, namely QoS values of composite services (short for QoS) and running times (short for RT) for different algorithms, in our experiments for performance evaluation.

5.2. Baseline Algorithms and Hyperparameters. To evaluate the performance of PPDRL-SC, we compare it with five state-of-the-art algorithms, namely multiconstrained optimal path problem for multistage graph (MCOP_M) [10], genetic algorithm (GA) [24], pointer network (PTR) [44], Q-learning (QLR) [36], and deep Q-learning (DQN) [3]. We provide a brief description for each algorithm as follows.

MCOP_M is a two-stage algorithm based on branch-and-bound strategy. It attempts to find a feasible service composition solution subject to multiple constraints simultaneously and maximize the utility of the solution.

GA is a genetic algorithm-based approach to solve the QSC problem. It uses the integer array encoding strategy, the standard two-points crossover operator, the random mutation operator, and the elitist selection method.

PTR is a framework to tackle combinatorial optimization problems using neural networks and reinforcement learning.

QLR applies reinforcement learning to obtain the optimal solution at runtime by directly studying the results of execution.

DQN decomposes an MOP into a number of single-objective optimization subproblems. At each iteration, a

Require: cs : the given composite service, C : the service class for cs , T : training steps, B : batch size.
Ensure: the optimal QoS value for cs
Initialize neural network params θ ;
Generate initial samples;
while convergence condition is not satisfied **do**
Update samples with better results if not the first cycle;
Pretrain the neural network based on MLE;
for $t = 1$ to T **do**
Given s_0 and θ , get the candidate services score distribution S ;
 $\{\pi_1, \pi_2, \dots, \pi_B\} \leftarrow \text{Sample}(S)$;
 $g_\theta \leftarrow 1/B \sum QoS(\pi_i | s_0) \nabla_{\theta} \log p_{\theta}(\pi_i | s_0)$
 $\theta \leftarrow \text{Adam}(\theta, g_\theta)$;
end for
end while

ALGORITHM 1: $PPDRL(cs, C, T, B)$.

TABLE 2: Hyperparameters for each algorithm.

Algorithm	Hyperparameters
MCOP_M	N/A
GA	population_size: 64, cross_rate: 0.5, mutation_rate: 0.2
PTR	embedding_hidden_dim: 128, LSTM_hidden_dim: 128, lr1_decay_step: 1000, lr1_decay_rate: 0.96, α : 0.9
QLR	learning_rate: 0.2, reward_decay: 0.9, e_greedy: 0.6
DQN	hidden_dim: 30, lr_start: 0.0002, lr_decay_step: 5000, lr_decay_rate: 0.96, max_epsilon: 0.9, min_epsilon: 0.1, epsilon_increment: 0.01, memory_capacity: 300
PPDRL	preipotrain_lr: 0.001, rl_lr: 0.0001, hidden_dim: 128

TABLE 3: The QoS values (mean and variance) on six different test cases.

Problem #	PPDRL		MCOP_M		GA		PTR		QLR		DQN	
	Mean	Var	Mean	Var	Mean	Var	Mean	Var	Mean	Var	Mean	Var
Nodes 10	0.32	0.00	0.24	$2.22e-3$	0.32	0.0000	0.29	$6.83e-5$	0.29	$1.19e-4$	0.25	$2.45e-4$
Nodes 30	0.31	0.00	0.13	$4.23e-3$	0.28	$4.38e-3$	0.13	$1.55e-5$	0.12	$5.54e-5$	0.05	$1.44e-4$
Nodes 50	0.20	0.0000	0.08	$1.37e-3$	0.20	$1.07e-7$	0.13	$7.12e-6$	0.13	$4.83e-6$	0.06	$3.32e-4$
Nodes 70	0.15	$6.64e-12$	0.06	$2.89e-4$	0.14	$1.02e-6$	0.07	$1.49e-5$	0.07	$2.22e-5$	0.02	$1.72e-4$
Nodes 90	0.14	0.0000	0.05	$4.30e-5$	0.13	$4.56e-6$	0.04	$6.50e-5$	0.03	$7.36e-5$	-0.02	$3.48e-5$
Nodes 100	0.20	$7.54e-7$	0.09	$4.30e-4$	0.18	$2.31e-6$	0.09	$2.57e-5$	0.08	$2.31e-4$	0.05	$5.18e-4$

predictive distribution model is built for each individual objective in the MOP by using fuzzy clustering and Gaussian stochastic process modeling.

Note that MCOP_M belongs to the classic algorithm category; GA is an mainstream evolutionary algorithm; and PTR, QLR, and DQN are three state-of-the-art reinforcement learning-based algorithm for the QSC problem. For a fair comparison, we adopt the recommended hyperparameter settings for the algorithms that have achieved the best performance reported in the previous literature, and the details are given in Table 2.

5.3. Experimental Results. The statistical results of the QoS values and the corresponding variances achieved by the six algorithms on six different test cases are summarized in

Table 3, where the best results are highlighted. We can see from Table 3 that PPDRL outperforms all the other five algorithms on six test cases. For some test cases (i.e., Nodes 30, 70, 90, 100), PPDRL can give significantly better solutions; for the other test cases (i.e., Nodes 10 and 50), PPDRL and GA can give similar solutions. These results indicate that PPDRL can give better mean solution quality in general. Furthermore, PPDRL gives smaller standard deviation of function values than other algorithms in most cases, and hence it has a more stable solution quality.

The detailed execution traces for the second run of six test cases are recorded and shown in Figure 2. In particular, the QoS values are listed with different running times. We can tell from Figure 2 that PPDRL obtains good initial QoS values compared to other algorithms. In addition, PPDRL requires relatively smaller number of iterations to converge

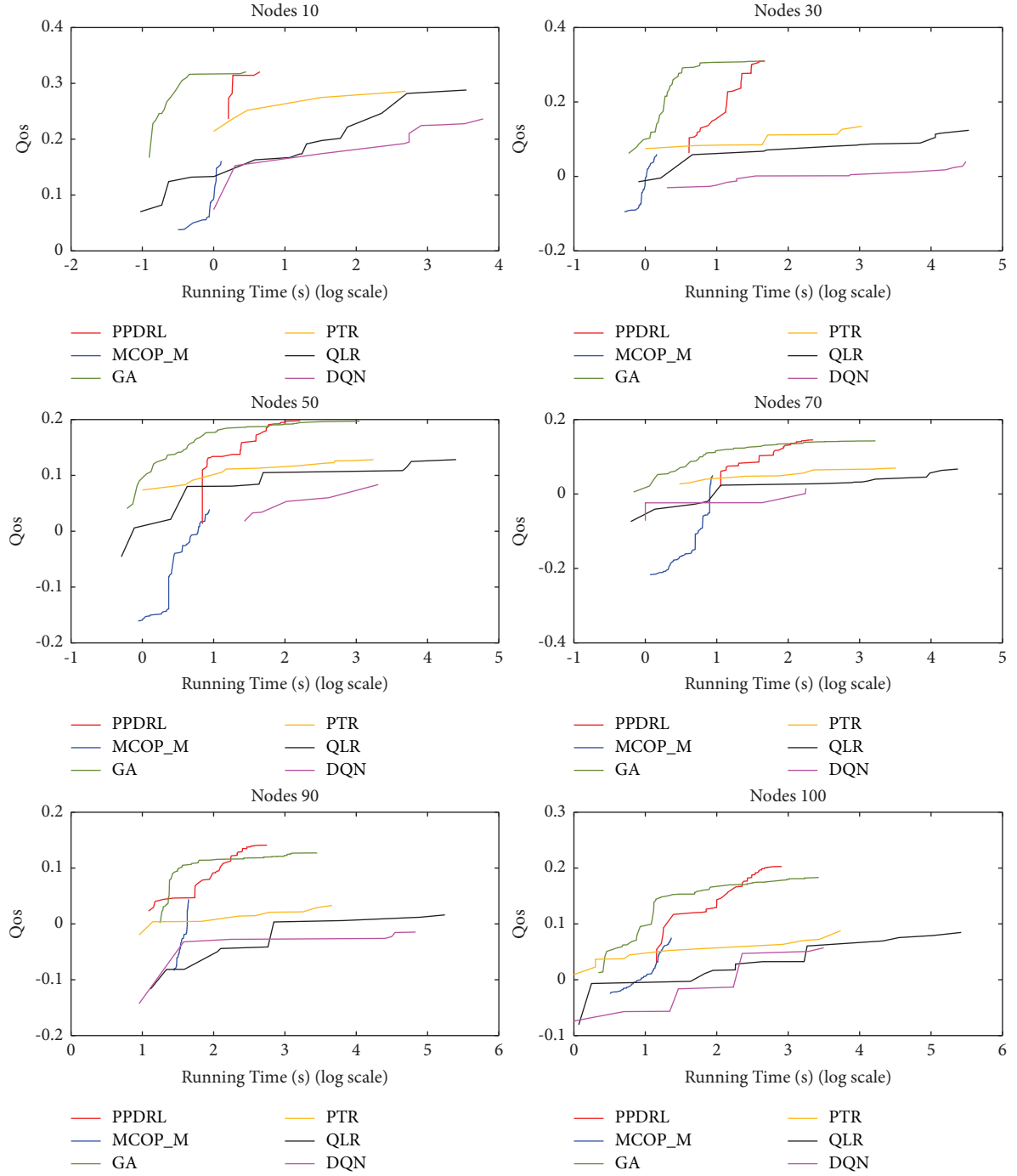


FIGURE 2: The QoS values over time (log scale).

than most of other algorithms, and it hence has a faster convergence speed.

Finally, we plot the QoS values of MCOP_M, GA, PTR, QLR, DQN, and PPDRL on six different test cases in Figure 3. In each figure, x -axis lists the test cases and y -axis represents the QoS values. We observe that PPDRL achieves an average of 129.00% improvement over MCOP_M, 6.11% improvement over GA, 114.86% improvement over PTR, 142.25% improvement over QLR, and 421.89% improvement over DQN. We can conclude from Figure 3 that PPDRL achieves stable and significant improvements

compared with the other five algorithms. During online training, PPDRL can converge to a better solution faster than other algorithms, which also indicates that PPDRL can adapt to a new network topology and service requests quickly. Another interesting observation is that the GA achieves surprisingly good results in our experiments. This is consistent with the findings of Julia in [8].

5.4. Threats to Validity. *Internal validity:* to increase the internal validity, we performed controlled experiments by

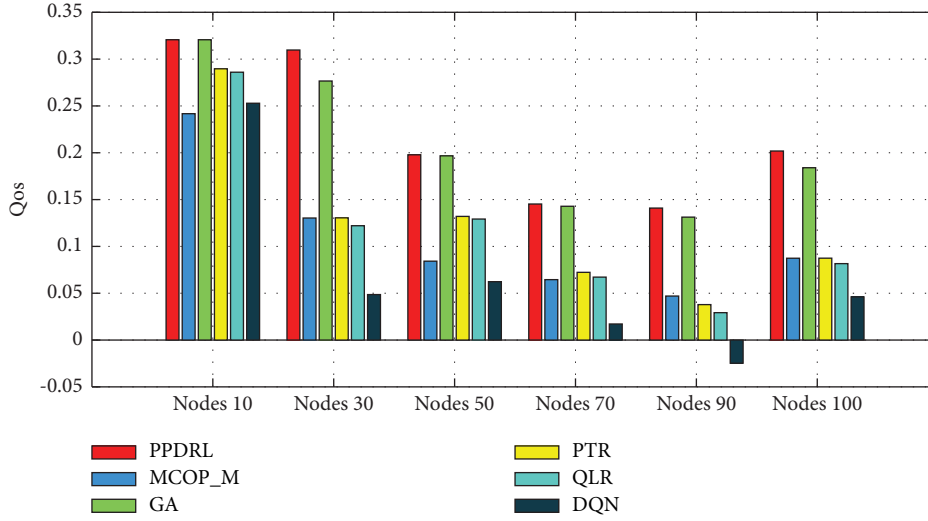


FIGURE 3: The QoS values for different algorithms on six test cases.

executing each test case five times and calculate the average of these five runs. Such method can avoid misleading effects of specifically selected test cases and ensures the stability of the results. In addition, we set the values of hyperparameters for each compared algorithm as suggested by their authors (see Table 2). Finally, we have tried multiple values of hyperparameters for PPDRL in our experiments and observed that the good values of these hyperparameters that can lead to better results are almost the same from test case to test case.

External validity: we increase the external validity by choosing six different composite service with different sizes. Furthermore, we are aware that because our PPDRL is a general black-box approach and is independent to the composite service structures and service classes, the results of our evaluations are transferable to other different service composition scenarios.

6. Conclusion and Future Work

In this paper, we propose PPDRL—a novel service composition solution based on deep reinforcement learning with pretraining-and-policy strategy for adaptive and large-scale service composition problems. The key idea of PPDRL is to incorporate a maximum likelihood estimate and a policy scoring mechanism into a deep reinforcement learning framework. More specifically, PPDRL uses the MLE to pretrain the neural network in order to accelerate convergence. After that, it uses the neural network to score the candidate atomic services for each atomic service and samples the candidate service set for each service multiple times based on the scores to prepare for the mini-batch gradient descent. As a result, such approach balances exploration and utilization and improves the efficiency of the algorithm. In depth experiments on PPDRL demonstrate its superior performance to five state-of-the-art optimization algorithms on six different testing scenarios.

Our further work includes refining our PPDRL approach by supporting the automatic selection of the appropriate

hyperparameters given a specific composite service and the corresponding service classes. We would compare our algorithms with latest benchmarks and with different network topology and service request. More performance metrics would be evaluated in our future work. In addition, we hope to abstract the proposed algorithm and release it as an automatic tool for different service composition scenarios.

Data Availability

No data were used to support this study

Conflicts of Interest

The authors declare that they have no conflicts of interest.

References

- [1] C. Jatoth, G. R. Gangadharan, U. Fiore, and R. Buyya, “Qos-aware big service composition using mapreduce based evolutionary algorithm with guided mutation,” *Future Generation Computer Systems*, vol. 86, pp. 1008–1018, 2018.
- [2] A. S. da Silva, E. Moshi, H. Ma, and S. Hartmann, “A qos-aware web service composition approach based on genetic programming and graph databases,” in *Proceedings of the International Conference on Database and Expert Systems Applications*, pp. 37–44, Springer, Bratislava, Slovakia, September 2017.
- [3] H. Wang, M. Gu, Q. Yu, H. Fei, J. Li, and Y. Tao, “Large-scale and adaptive service composition using deep reinforcement learning,” in *Proceedings of the International Conference on Service-Oriented Computing*, pp. 383–391, Springer, Malaga, Spain, November 2017.
- [4] S. Chattopadhyay and A. Banerjee, “Qos constrained large scale web service composition using abstraction refinement,” *IEEE Transactions on Services Computing*, vol. 13, 2017.
- [5] Y. Shi and Xi Chen, “A survey on qos-aware web service composition,” in *Proceedings of the 2011 Third International Conference on Multimedia Information Networking and Security*, pp. 283–287, IEEE, Shanghai, China, November 2011.

- [6] M. W. Y. Mao, T. Ben, and R. Li, "Service importance classification method based on service state perception," *Command Information System and Technology*, vol. 11, no. 3, 2020.
- [7] H. D. Xia zhou, "Design and implementation of distributed service framework," *Command Information System and Technology*, vol. 11, no. 1, 2020.
- [8] J. Amin, E. Sundararajan, and Z. Othman, "Cloud computing service composition: a systematic literature review," *Expert Systems with Applications*, vol. 41, no. 8, pp. 3809–3824, 2014.
- [9] J. J.-W. Yoo, S. Kumara, D. Lee, and S.-C. Oh, "A web service composition framework using integer programming with non-functional objectives and constraints," in *Proceedings of the 2008 10th IEEE Conference on E-Commerce Technology and the Fifth IEEE Conference on Enterprise Computing, E-Commerce and E-Services*, pp. 347–350, IEEE, Arlington, VA, USA, July 2008.
- [10] D. Wang, Y. Yang, and Z. Mi, "Qos-based and network-aware web service composition across cloud datacenters," *KSII Transactions on Internet & Information Systems*, vol. 9, no. 3, 2015.
- [11] S.-L. Fan, K.-Y. Peng, and Y.-B. Yang, "Large-scale qos-aware service composition integrating chained dynamic programming and hybrid pruning," in *Proceedings of the International Conference on Web Services*, pp. 196–211, Springer, San Francisco, CA, USA, July 2018.
- [12] G. White, A. Palade, and S. Clarke, "Qos prediction for reliable service composition in iot," in *Proceedings of the International Conference on Service-Oriented Computing*, pp. 149–160, Springer, Malaga, Spain, November 2017.
- [13] A. A. Wakrime and S. Jabbour, "Formal approach for qos-aware cloud service composition," in *Proceedings of the IEEE 26th International Conference on Enabling Technologies: Infrastructure for Collaborative Enterprises (WETICE)*, pp. 30–35, IEEE, Poznan, Poland, June 2017.
- [14] Y. Yan and M. Chen, "Anytime qos-aware service composition over the graphplan," *Service oriented computing and applications*, vol. 9, no. 1, pp. 1–19, 2015.
- [15] R. Mirandola, D. Perez-Palacin, P. Scandurra, M. Brignoli, and A. Zonca, "Business process adaptability metrics for qos-based service compositions," in *Proceedings of the European Conference on Service-Oriented and Cloud Computing*, pp. 110–124, Springer, Taormina, Italy, September 2015.
- [16] S. B. Bhushan and P. C. H. Reddy, "A hybrid meta-heuristic approach for qos-aware cloud service composition," *International Journal of Web Services Research*, vol. 15, no. 2, pp. 1–20, 2018.
- [17] P. Rodriguez-Mier, M. Mucientes, and M. Lama, "Hybrid optimization algorithm for large-scale qos-aware service composition," *IEEE transactions on services computing*, vol. 10, no. 4, pp. 547–559, 2017.
- [18] T. H. A. S. Siriweera, I. Paik, and B. T. G. S. Kumara, "Qos and customizable transaction-aware selection for big data analytics on automatic service composition," in *Proceedings of the 2017 IEEE International Conference on Services Computing (SCC)*, pp. 116–123, IEEE, Honolulu, HI, USA, June 2017.
- [19] S. R. Boussalia, A. Chaoui, and A. Hurault, "Qos-based web services composition optimization with an extended bat inspired algorithm," in *Proceedings of the International Conference on Information and Software Technologies*, pp. 306–319, Springer, Druskininkai, Lithuania, October 2015.
- [20] O. Hammas, S. Ben Yahia, and S. Ben Ahmed, "Adaptive web service composition insuring global qos optimization," in *Proceedings of the 2015 International Symposium on Networks, Computers and Communications (ISNCC)*, pp. 1–6, IEEE, Yasmine Hammamet, Tunisia, May 2015.
- [21] H. Wang, X. Xu, Z. Liu, and Z. Wang, "The configurability study on artificial bee colony algorithm for qos-aware service composition," in *Proceedings of the 2015 International Conference on Service Science (ICSS)*, pp. 106–112, IEEE, Weihai, China, May 2015.
- [22] Q. Wu and Q. Zhu, "Transactional and qos-aware dynamic service composition based on ant colony optimization," *Future Generation Computer Systems*, vol. 29, no. 5, pp. 1112–1119, 2013.
- [23] A. Klein, F. Ishikawa, and S. Honiden, "Efficient heuristic approach with improved time complexity for qos-aware service composition," in *Proceedings of the 2011 IEEE International Conference on Web Services*, pp. 436–443, IEEE, Washington, DC, USA, July 2011.
- [24] G. Canfora, M. Di Penta, R. Esposito, and M. Luisa Villani, "An approach for qos-aware service composition based on genetic algorithms," in *Proceedings of the 7th annual conference on Genetic and evolutionary computation*, pp. 1069–1075, ACM, Washington, DC, USA, June 2005.
- [25] F. Bari, S. R. Chowdhury, R. Ahmed, R. Boutaba, and O. C. M. B. Duarte, "Orchestrating virtualized network functions," *IEEE Transactions on Network and Service Management*, vol. 13, no. 4, pp. 725–739, 2016.
- [26] J. Liu, W. Lu, F. Zhou, P. Lu, and Z. Zhu, "On dynamic service function chain deployment and readjustment," *IEEE Transactions on Network and Service Management*, vol. 14, no. 3, pp. 543–553, 2017.
- [27] H. Labbaci, B. Medjahed, and Y. Aklouf, "A deep learning approach for long term qos-compliant service composition," in *Proceedings of the International Conference on Service-Oriented Computing*, pp. 287–294, Springer, Malaga, Spain, November 2017.
- [28] A. A. Pourhaji Kazem, H. Pedram, and H. Abolhassani, "Bnqm: a bayesian network based qos model for grid service composition," *Expert Systems with Applications*, vol. 42, no. 20, pp. 6828–6843, 2015.
- [29] H. Wang, Q. Wu, X. Chen, and Q. Yu, "Integrating Gaussian process with reinforcement learning for adaptive service composition," in *Proceedings of the International Conference on Service-Oriented Computing*, pp. 203–217, Springer, Goa, India, November 2015.
- [30] H. Wang, X. Chen, Q. Wu, Q. Yu, Z. Zheng, and A. Bouguettaya, "Integrating on-policy reinforcement learning with multi-agent techniques for adaptive service composition," in *Proceedings of the International Conference on Service-Oriented Computing*, pp. 154–168, Springer, Paris, France, November 2014.
- [31] H. Wang, G. Huang, and Y. Qi, "Automatic hierarchical reinforcement learning for efficient large-scale service composition," in *Proceedings of the 2016 IEEE International Conference on Web Services (ICWS)*, pp. 57–64, IEEE, San Francisco, CA, USA, July 2016.
- [32] J. Alexander, F. Mohr, and B. Kleinjohann, "Applying reinforcement learning for resolving ambiguity in service composition," in *Proceedings of the 2014 IEEE 7th International Conference on Service-Oriented Computing and Applications*, pp. 105–112, IEEE, Matsue, Japan, November 2014.
- [33] D. H. Elsayed, E. S. Nasr, A. E. D. M. El Ghazali, and M. H. Gheith, "A new hybrid approach using genetic algorithm and q-learning for qos-aware web service composition," in *Proceedings of the International Conference on Advanced*

- Intelligent Systems and Informatics*, pp. 537–546, Springer, Cairo, Egypt, September 2017.
- [34] Y. Zhao, S. Wang, Y. Zou, J. Ng, and T. Ng, “Automatically learning user preferences for personalized service composition,” in *Proceedings of the 2017 IEEE International Conference on Web Services (ICWS)*, pp. 776–783, IEEE, Honolulu, HI, USA, June 2017.
 - [35] U. G. Shehu, *Novel Optimization Schemes for Service Composition in the Cloud Using Learning Automata-Based Matrix Factorization*, University of Bedfordshire, Luton, UK, 2015.
 - [36] H. Wang, X. Zhou, X. Zhou, W. Liu, W. Li, and A. Bouguettaya, “Adaptive service composition based on reinforcement learning,” in *Service-Oriented Computing*, pp. 92–107, Springer, Berlin, Germany, 2010.
 - [37] B. Li and Z. Zhu, “Gnn-based hierarchical deep reinforcement learning for nfv-oriented online resource orchestration in elastic optical dcis,” *Journal of Lightwave Technology*, vol. 40, no. 4, pp. 935–946, 2022.
 - [38] B. Li, W. Lu, and Z. Zhu, “Deep-nfvorch: leveraging deep reinforcement learning to achieve adaptive vnf service chaining in dci-eons,” *Journal of Optical Communications and Networking*, vol. 12, no. 1, 2020.
 - [39] B. Liang, F. Zhao, M. Shen, Y. Qi, and P. Chen, “An orthogonal genetic algorithm for qos-aware service composition,” *The Computer Journal*, vol. 59, no. 12, pp. 1857–1871, 2016.
 - [40] B. Madan and D. Bein, “Optimal maximum likelihood estimates fusion in distributed network of sensors,” in *Proceedings of the 2016 IEEE 12th International Conference on Intelligent Computer Communication and Processing (ICCP)*, pp. 369–375, IEEE, Cluj-Napoca, Romania, September 2016.
 - [41] D. P. Kingma and B. Jimmy, “Adam: a method for stochastic optimization,” 2014, <https://arxiv.org/abs/1412.6980>.
 - [42] G. Juve, A. Chervenak, E. Deelman, S. Bharathi, G. Mehta, and K. Vahi, “Characterizing and profiling scientific workflows,” *Future Generation Computer Systems*, vol. 29, no. 3, pp. 682–692, 2013.
 - [43] E. Al-Masri and Q. H. Mahmoud, “Discovering the best web service,” in *Proceedings of the 16th international conference on World Wide Web*, pp. 1257–1258, ACM, Banff Alberta Canada, May 2007.
 - [44] I. Bello, H. Pham, V. L. Quoc, M. Norouzi, and S. Bengio, “Neural Combinatorial Optimization with Reinforcement Learning,” 2016, <https://arxiv.org/abs/1611.09940>.